

PRESIDENTIAL GRADUATE SCHOOL

Course: Knowledge Discovery and Data Science (BSIT-375)

Week 8: Final Project Assignment

Nepse-Sight

*A Real-Time Stock Market Intelligence and Prediction Platform
for NEPSE Sectors*

April 23, 2026

Project Overview

This project challenges you to build a comprehensive **real-time stock market analytics and prediction system** for the Nepal Stock Exchange (NEPSE). You will ingest historical data from at least five major sectors (Hydro, Banking, Manufacturing, Insurance, and Hotels), design a streaming pipeline to collect live price data during trading hours, perform exploratory data analysis (EDA), implement multiple forecasting models, and present insights through an interactive dashboard featuring a rich set of technical indicators. The system must be containerised and capable of updating visualisations in near real-time.

1 Project Objectives

By completing this assignment, students will:

- Acquire and preprocess **historical stock data** for at least five NEPSE sectors (Hydro, Banking, Manufacturing, Insurance, Hotels) covering a minimum of 5 years.
- Design a **real-time data ingestion service** that scrapes live price information (at least one representative company per sector) from sources like Nepse Alpha or direct NEPSE pages during trading hours (Monday–Friday, 11:00 AM – 3:00 PM NPT) at one-minute intervals.
- Build a **streaming data pipeline** using Apache Kafka to decouple data collection, processing, and model inference.
- Conduct **Exploratory Data Analysis (EDA)** on the combined historical and real-time datasets, generating statistical summaries, correlation matrices, and distribution plots.
- Implement and compare three **forecasting models**:
 - ARIMA (AutoRegressive Integrated Moving Average)
 - Multiple Linear Regression
 - Bi-directional LSTM (Long Short-Term Memory)
- Develop an **interactive dashboard** (using Tableau, Power BI, Grafana, or a custom web framework) that displays:
 - At least **10 technical charts** (e.g., candlestick patterns, moving averages, RSI, MACD, Bollinger Bands, volume analysis, etc.)
 - Real-time price updates for selected stocks
 - Model predictions with confidence intervals
- Containerise all services with Docker and orchestrate using Docker Compose, with optional distributed deployment across multiple nodes.

2 System Architecture Requirements

The pipeline follows a layered, event-driven design (Figure 1). Communication between services is asynchronous via Kafka topics.

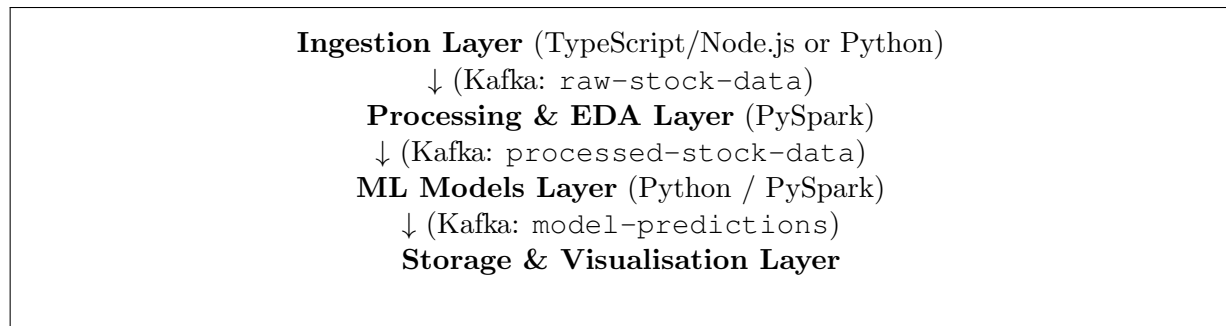


Figure 1: High-level pipeline architecture

2.1 Data Ingestion Service (Polling Layer)

- **Language:** TypeScript (Node.js) or Python.
- **Responsibilities:**
 - Historical Data Loading:** On startup, load CSV/Excel files containing daily OHLCV (Open, High, Low, Close, Volume) data for the required sectors and companies. Store this in the chosen database.
 - Real-time Scraping:** During trading hours, poll a reliable source (e.g., Npse Alpha, direct NEPSE page, or third-party aggregator) every minute. Extract current price, volume, and timestamp for at least one company from each of the five sectors.
 - Publish raw data as JSON messages to Kafka topic `raw-stock-data`.
 - Implement robust error handling and respect rate limits to avoid IP blocking.
 - Provide configuration for trading schedule and polling interval.

2.2 Data Preprocessing & EDA Layer

- **Language & Framework:** Python with **PySpark** (Structured Streaming).
- **Responsibilities:**
 - Consume messages from `raw-stock-data`.
 - Clean and validate data: handle missing values, detect outliers, and convert timestamps to NPT.
 - Merge real-time ticks with historical data to maintain a continuous time series.
 - Compute technical indicators (e.g., moving averages, RSI, MACD, Bollinger Bands) in a streaming fashion using windowed aggregations.
 - Publish cleaned and enriched data to Kafka topic `processed-stock-data`.
 - EDA:** Generate summary statistics, correlation heatmaps, and distribution plots (as static reports or via Jupyter notebooks) using the historical dataset and initial streaming window.

2.3 ML Models Service (Prediction Layer)

- **Language & Framework:** Python (with libraries like statsmodels, scikit-learn, TensorFlow/PyTorch).
- **Responsibilities:**
 - a. Train (or update) three forecasting models on historical data:
 - **ARIMA:** For univariate time-series forecasting of closing prices.
 - **Multiple Linear Regression:** Using lagged prices and technical indicators as features.
 - **Bi-directional LSTM:** A deep learning model that captures long-term dependencies in both forward and backward directions.
 - b. Consume processed data from `processed-stock-data` and generate short-term forecasts (e.g., next 5, 15, 30 minutes) for each tracked stock.
 - c. Publish predictions (model name, forecasted values, confidence intervals) to Kafka topic `model-predictions`.
 - d. Periodically retrain models (e.g., daily or after market close) to incorporate new data.

2.4 Storage & Visualisation Layer

- **Storage:** Use a time-series database (e.g., InfluxDB) or a relational database (PostgreSQL with TimescaleDB) to efficiently store historical and real-time stock data.
- **Dashboard:**
 - **Mandatory Technical Charts (minimum 10 distinct types):**
 1. Candlestick chart with volume
 2. Simple Moving Average (SMA) and Exponential Moving Average (EMA) overlays
 3. Relative Strength Index (RSI)
 4. Moving Average Convergence Divergence (MACD)
 5. Bollinger Bands
 6. On-Balance Volume (OBV)
 7. Stochastic Oscillator
 8. Average True Range (ATR)
 9. Ichimoku Cloud (optional but encouraged)
 10. Price-by-Volume (PBV) chart
 11. Model forecast vs. actual price line chart
 12. Sector performance comparison (bar/heatmap)
 - **Tools:** You may use Tableau, Power BI, Grafana, or build a custom dashboard with Streamlit / Dash / React.
 - The dashboard must refresh automatically (every 1–5 minutes) to reflect new incoming data and updated predictions.

3 Technology Stack & Constraints

Component	Allowed Technologies	Notes
Ingestion	Node.js / TypeScript or Python	Containerised
Message Broker	Apache Kafka	Use Confluent Docker images
Processing & EDA	PySpark (Python)	Structured Streaming
ML Models	Python (statsmodels, sklearn, Keras/PyTorch)	ARIMA, Linear Regression, Bi-LSTM
Database	InfluxDB / PostgreSQL+TimescaleDB	Time-series optimised
Visualisation	Tableau / Power BI / Grafana / Streamlit	Must connect to database
Orchestration	Docker Compose	Single <code>docker-compose.yml</code>

tableTechnology stack and constraints

Additional Requirements:

- The entire system must be runnable via `docker-compose up`.
- Provide a `README.md` with setup instructions, configuration details (trading hours, scraped URLs, companies list), and an architecture diagram.
- For distributed deployment (multiple laptops/nodes), students may use Docker Swarm or Kubernetes – document the steps.
- **Collaboration Encouraged:** You are permitted and encouraged to consult with BBA/MBA students or faculty to better understand stock market technical analysis and chart interpretation. Properly acknowledge any assistance received.

4 Data Sources and Companies

Students must identify and scrape at least one actively traded company from each sector. Below are examples (you may choose others):

Sector	Example Company (Symbol)
Hydro Power	Butwal Power Company (BPCL)
Banking	Nabil Bank (NABIL)
Manufacturing & Processing	Unilever Nepal (UNL)
Insurance	Nepal Life Insurance (NLIC)
Hotels & Tourism	Soaltee Hotel (SHL)

Table 1: Suggested companies per sector

Historical data can be obtained from:

- Nipse Alpha (<https://nipsealpha.com>)
- Sharesansar (<https://sharesansar.com>)
- Direct NEPSE website (requires careful scraping)

5 Project Presentation Guidelines

1. **Source Code:** A GitHub repository containing the complete codebase.
2. **Documentation:** A README file describing the setup instructions and the architecture diagram.
3. **Demonstration:** A live demo showing data flowing from a news portal to the dashboard in real-time.

May your predictions be accurate and your charts insightful!